

Higher-order Functions

Übungen

Combiner

Gegeben ist folgender Unit Test. **Implementiere** das FunctionalInterface **Combiner** passend zum Unit Test. Der Unit Test darf nicht verändert werden!

```
import org.junit.jupiter.api.Test;

import static com.google.common.truth.Truth.assertThat;

class CombinerTest {

    private static final double TOLERANCE = 0.000000000001;

    @Test
    void stringsWithSpaceCombiner() {
        Combiner<String, String> withSpaceCombiner = (string1, string2) -> "%s %s".formatted(string1, string2);

        String actual = withSpaceCombiner.combine("Elon", "Musk");

        assertThat(actual).isEqualTo("Elon Musk");
    }

    @Test
    void integersAsStringCombiner() {
        Combiner<Integer, String> numbersAsStringCombiner = (number1, number2) -> "%s%s".formatted(number1, number2);

        String actual = numbersAsStringCombiner.combine(4, 2);

        assertThat(actual).isEqualTo("42");
    }

    @Test
    void integersAsDoubleCombiner() {
        Combiner<Integer, Double> numbersAsStringCombiner = (number1, number2) -> (double) (number1 + number2);

        Double actual = numbersAsStringCombiner.combine(4, 2);

        assertThat(actual).isWithin(TOLERANCE).of(6.0);
    }
}
```

Combiner: Lösung

```
@FunctionalInterface  
interface Combiner<T, R> {  
  
    R combine(T element1, T element2);  
}
```

MovieDatabase

Gegeben sind folgende Klassen inklusive Unit Test. **Implementiere** die Klasse **MovieDatabase** passend zum Unit Test (siehe fettgedruckte Methoden), aber **ohne Streams**. Der Unit Test, Movie, und Category dürfen nicht verändert werden!

```
import java.util.Arrays;
import java.util.EnumSet;
import java.util.Objects;
import java.util.Set;

class Movie {
    private final String name;
    private final Set<Category> categories;

    Movie(String name, Category... categories) {
        this.name = name;
        this.categories = EnumSet.copyOf(Arrays.asList(categories));
    }

    String name() {
        return name;
    }

    boolean hasCategory(Category category) {
        return categories.contains(category);
    }

    @Override
    public final boolean equals(Object object) {
        if (this == object) return true;
        if (!(object instanceof Movie movie)) return false;
        return Objects.equals(name, movie.name) &&
            Objects.equals(categories, movie.categories);
    }

    @Override
    public int hashCode() {
        int result = Objects.hashCode(name);
        result = 31 * result + Objects.hashCode(categories);
        return result;
    }

    @Override
    public String toString() {
        return "%s %s".formatted(name, categories);
    }
}
```

```
enum Category {
    HORROR, ROMANCE, ACTION, SCIFI, FANTASY, COMEDY, THRILLER, DRAMA
}
```

```
import org.junit.jupiter.api.Test;
import java.util.*;
import static com.google.common.truth.Truth.assertThat;

class MovieDatabaseTest {

    private static final List<Movie> MOVIES = List.of(
        new Movie("The Godfather", Category.DRAMA),
        new Movie("Toy Story", Category.COMEDY),
        new Movie("Guardians of the Galaxy", Category.ACTION, Category.COMEDY, Category.SCIFI),
        new Movie("Titanic", Category.DRAMA, Category.ROMANCE),
        new Movie("The Dark Knight", Category.ACTION, Category.DRAMA),
        new Movie("Finding Nemo", Category.COMEDY, Category.DRAMA),
        new Movie("Interstellar", Category.SCIFI, Category.DRAMA),
        new Movie("Alien", Category.SCIFI, Category.HORROR),
        new Movie("Terminator", Category.SCIFI, Category.ACTION),
        new Movie("Dune", Category.SCIFI, Category.ACTION, Category.DRAMA)
    );

    private final MovieDatabase movieDatabase = new MovieDatabase(MOVIES); // inject test movies into the movie database.

    @Test
    void forEachCallsLambdaWithEachMovie() {
        List<Movie> movies = new LinkedList<>();

        movieDatabase.forEach(movie -> {
            movies.add(movie); // This is not a pure lambda, but okay for this testcase, otherwise forEach not testable.
            // Note: lambda returns void
        });

        assertThat(movies).containsExactlyElementsIn(MOVIES);
    }

    @Test
    void findMoviesByCategory() {
        Collection<Movie> actual = movieDatabase.findByCategory(movie -> movie.hasCategory(Category.SCIFI) &&
            movie.hasCategory(Category.ACTION));

        assertThat(actual).containsExactlyElementsIn(List.of(
            new Movie("Guardians of the Galaxy", Category.ACTION, Category.SCIFI, Category.COMEDY),
            new Movie("Terminator", Category.ACTION, Category.SCIFI),
            new Movie("Dune", Category.ACTION, Category.SCIFI, Category.DRAMA)
        ));
    }
}
```

MovieDatabase: Lösung

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.function.Consumer;
import java.util.function.Predicate;

class MovieDatabase {

    private final Collection<Movie> movies;

    MovieDatabase(Collection<Movie> movies) {
        this.movies = movies;
    }

    void forEach(Consumer<Movie> action) {
        for (Movie movie : movies) {
            action.accept(movie);
        }

        // Alternativ:
        // return movies.stream()
        //     .forEach(action);
    }

    Collection<Movie> findByCategory(Predicate<Movie> criteria) {
        Collection<Movie> result = new ArrayList<>();
        for (Movie movie : movies) {
            if (criteria.test(movie)) {
                result.add(movie);
            }
        }
        return result;

        // Alternative mit Stream:
        // return movies.stream()
        //     .filter(criteria)
        //     .toList();
    }
}
```

MyList

MyList stellt eine eigene, vereinfachte Implementierung einer Liste dar (wie z. B. ArrayList). Allerdings enthält MyList lediglich einen **Konstruktor** und eine **contains-Methode**. Gegeben ist folgender Unit Test. **Implementiere** die Klasse **MyList** passend zum Unit Test, aber **ohne Streams**. Der Unit Test darf nicht verändert werden!

```
import org.junit.jupiter.api.Test;
import java.util.List;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

class MyListTest {

    // Der Konstruktor in MyList übernimmt und speichert die vier übergebenen Elemente. D. h. myList enthält anschließend die vier Strings.
    private final MyList<String> myList = new MyList<>{List.of("Elon", "Sara", "Jeff", "Mark")};

    // contains mit Objekt -----

    @Test
    void containsTrue() {
        assertTrue(myList.contains("Elon"));
    }

    @Test
    void containsFalse() {
        assertFalse(myList.contains("leon"));
    }

    @Test
    void containsFalseCaseSensitive() {
        assertFalse(myList.contains("elon"));
    }

    // contains mit Lambda -----

    @Test
    void containsLambdaTrue() {
        assertTrue(myList.contains(element -> element.equals("Elon")));
    }

    @Test
    void containsLambdaTrueCaseInsensitive() {
        assertTrue(myList.contains(element -> element.equalsIgnoreCase("elon")));
    }

    @Test
    void containsLambdaFalse() {
        assertFalse(myList.contains(element -> element.equals("leon")));
    }

    // contains mit Lambda, aber anderer Typ als String -----

    @Test
    void containsOtherTypeTrue() {
        var myIntegerList = new MyList<>{List.of(1, 2, 3)};
        assertTrue(myIntegerList.contains(element -> element.equals(3)));
    }

    @Test
    void containsOtherTypeFalse() {
        var myIntegerList = new MyList<>{List.of(1, 2, 3)};
        assertFalse(myIntegerList.contains(element -> element.equals(4)));
    }
}
```

MyList: Lösung

```
class MyList<T> {  
  
    private final List<T> elements;  
  
    MyList(Collection<T> elements) {  
        this.elements = new ArrayList<>(elements); // create copy of elements received from parameter.  
    }  
  
    boolean contains(T element) {  
        return elements.contains(element);  
    }  
  
    boolean contains(Predicate<T> criteria) {  
        for (T element : elements) {  
            if (criteria.test(element)) {  
                return true;  
            }  
        }  
        return false;  
    }  
  
    // Alternative mit Streams:  
    // return elements.stream().anyMatch(criteria);  
}
```

MyList (Bonusaufgabe)

Füge einen weiteren Konstruktor zur Klasse MyList hinzu, der Varargs als Parameter akzeptiert:

T... elements

Auch der neue Konstruktor soll die übergebenen Elemente in die interne Liste von MyList kopieren. #DRY

Erweitere nun auch den Unit Test um Testfälle, sodass auch der neue Konstruktor getestet wird.

MyList (Bonusaufgabe): Lösung

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.List;
import java.util.function.Predicate;

class MyList<T> {

    private final List<T> elements;

    MyList(Collection<T> elements) {
        this.elements = new ArrayList<>(elements); // create copy of elements received from parameter.
    }

    // Bonusaufgabe
    @SafeVarargs
    MyList(T... elements) {
        this(Arrays.asList(elements)); // re-use other constructor. #DRY
    }

    // ...
}
```

```
class MyListTest {

    // ...

    // Bonusaufgabe: Varargs mit T... elements
    @Test
    void containsWithVarArgsTrue() {
        var myList = new MyList<>("Elon", "Sara", "Jeff");
        assertTrue(myList.contains("Elon"));
    }

    @Test
    void containsWithVarArgsFalse() {
        var myList = new MyList<>("Elon", "Sara");
        assertFalse(myList.contains("leon"));
    }
}
```

HigherOrderUtil

Gegeben ist folgender Unit Test. **Implementiere** die Klasse **HigherOrderUtil** passend zum Unit Test. Der Unit Test darf nicht verändert werden!

Hinweis: Die Methoden `applyToEach` und `applyToEachIf` verwenden **ausschließlich Integer (keine Generics oder andere primitive Datentypen)**.

```
import org.junit.jupiter.api.Test;
import java.util.List;
import static com.google.common.truth.Truth.assertThat;

class HigherOrderUtilTest {

    @Test
    void applyToEach() {
        List<Integer> numbers = List.of(1, 2, 3, 4, 5);

        // returns each modified number based on the given lambda.
        List<Integer> doubled = HigherOrderUtil.applyToEach(numbers, x -> x * 2); // Parameters: 1st source, 2nd modifier lambda

        assertThat(doubled).isEqualTo(List.of(2, 4, 6, 8, 10));
    }

    @Test
    void applyToEachIf() {
        List<Integer> numbers = List.of(1, 2, 3, 4, 5);

        // returns modified numbers, but only for those numbers that fulfil a given condition (3rd parameter).
        List<Integer> halfedIfEven = HigherOrderUtil.applyToEachIf(numbers, x -> x / 2, x -> x % 2 == 0); // Parameters: 1st source, 2nd modifier lambda, 3rd condition lambda

        assertThat(halfedIfEven).isEqualTo(List.of(1, 2));
    }
}
```

HigherOrderUtil: Lösung

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import java.util.function.Predicate;
import java.util.function.UnaryOperator;

final class HigherOrderUtil {

    static List<Integer> applyToEach(Collection<Integer> numbers, UnaryOperator<Integer> function) { // or: Function<Integer, Integer> function
        List<Integer> result = new ArrayList<>();
        for (Integer number : numbers) {
            Integer i = function.apply(number);
            result.add(i);
        }
        return result;

        // Alternative mit Streams:
        // return numbers.stream()
        // .map(function)
        // .toList();
    }

    static List<Integer> applyToEachIf(Collection<Integer> numbers, UnaryOperator<Integer> function, Predicate<Integer> condition) {
        List<Integer> result = new ArrayList<>();

        for (Integer number : numbers) {
            if (condition.test(number)) {
                Integer i = function.apply(number);
                result.add(i);
            }
        }

        return result;

        // Alternative mit Streams:
        // return numbers.stream()
        // .filter(condition)
        // .map(function)
        // .toList();
    }
}
```

HtmlGenerator

Gegeben ist folgender Unit Test. **Implementiere** die Klasse **HtmlGenerator** passend zum den Unit Test. Der Unit Test darf nicht verändert werden!

Hinweis: In Java werden mehrzeilige Strings mit `"""` geschrieben. Achtung: Der mehrzeilige Text darf nicht in derselben Zeile stehen wie das einleitende `"""`.

```
import org.junit.jupiter.api.Test;
import static com.google.common.truth.Truth.assertThat;

class HtmlGeneratorTest {
```

```
    @Test
    void generate() {
        String htmlDocument = new HtmlGenerator() // Method Chaining
            .setLanguage(Language.GERMAN)
            .setTitle(language -> titleTranslator(language))
            .setBody(() -> """
                <h1>Hello Elon!</h1>
                <a href="impressum">Impressum</a>""")
            .generate();
```

```
        assertThat(htmlDocument).isEqualTo("""
            <!DOCTYPE html>
            <html>
            <head>
            <title>Beste Seite der Welt</title>
            </head>
            <body>
            <h1>Hello Elon!</h1>
            <a href="impressum">Impressum</a>
            </body>
            </html>""");
    }
```

```
    private static String titleTranslator(Language language) {
        return switch (language) {
            case GERMAN -> "Beste Seite der Welt";
            case KLINGON -> "taghDI' qo' vIDa ";
            default -> "Best Page of the World";
        };
    }
}
```

```
enum Language {
    GERMAN,
    KLINGON,
    ENGLISH
}
```

Mehrzeiliger String: Der Text beginnt in der nächsten Zeile nach dem ersten `"""`.

Für jede neue Zeile fügt Java automatisch ein unsichtbares `\n` zum Text hinzu, welches einen Zeilenumbruch repräsentiert.

Hinweis:

Die Formatierung des HTML-Dokuments ist wichtig und muss identisch mit dem Erwartungswert sein! D. h., dass alle Zeilenumbrüche und führende Leerzeichen müssen identisch sein.

Tipp:

Verwende folgenden Code, um die einzelnen Zeilen in einem mehrzeiligen String zu erhalten:
`String[] lines = multiLineString.split("\n");` // Das `\n` ist ein Spezialzeichen und steht für Zeilenumbruch.

Tipp:

Implementiere am besten eine Methode nach der anderen. Kommentiere hierzu die restlichen Methoden so lange aus. Fange z. B. mit `generate()` an, welche den zusammengebauten String zurückgibt (den du nach und nach erweiterst).

HtmlGenerator: Lösung

```
import java.util.function.Function;
import java.util.function.Supplier;

class HtmlGenerator {

    private static final String LINE_BREAK = "\n";
    private static final String INDENT = "  ";

    // Setze Default-Werte, um mögliche Nullpointer-Exception zu vermeiden.
    private Function<Language, String> titleTranslator = _ -> "";
    private Supplier<String> body = () -> "";
    private Language language = Language.ENGLISH;

    HtmlGenerator setLanguage(Language language) {
        this.language = language;
        return this;
    }

    HtmlGenerator setTitle(Function<Language, String> titleTranslator) {
        this.titleTranslator = titleTranslator;
        return this;
    }

    // Hinweis: Higher-order Function ist für die Methode setBody ein wenig Overkill,
    // da prinzipiell ein einfacher String ausreichen würde.
    // Hier zu Übungszwecken so konstruiert, dass mit Supplier gearbeitet werden soll.
    HtmlGenerator setBody(Supplier<String> supplier) {
        body = supplier;
        return this;
    }

    String generate() {
        return startHtml() + generateHead() + generateBody() + closeHtml();
    }
}
```

```
private String startHtml() {
    return ""
        <!DOCTYPE html>
        <html>
        """;
}

private String closeHtml() {
    return "</html>";
}

private String generateHead() {
    return ""
        <head>
        <title>%s</title>
        </head>
        """.formatted(titleTranslator.apply(language));
}

private String generateBody() {
    StringBuilder result = new StringBuilder();

    result.append("<body>").append(LINE_BREAK);

    String[] lines = body.get().split(LINE_BREAK);
    for (String line : lines) {
        result.append(INDENT).append(line).append(LINE_BREAK);
    }

    result.append("</body>").append(LINE_BREAK);

    return result.toString();
}
}
```